CTF Challenge Write-up: "Trustworthy Person"

Category: Cryptography

I am really trustworthy!

Challenge Summary

"Trustworthy Person" is a cryptographic challenge centered around Elliptic Curve Cryptography (ECC). The server generates a secret key d and a public key Q on the secp256k1 curve (the same one used by Bitcoin).

```
└─$ nc chall.25.cuhkctf.org 25059
Ciallo～(∠· w< )⌒★
I know a secret number d, and I can tell you Q.
Q: Point(100394412836481808745691364052876198933983008869097666396622517336554667811129, 23422732540388099646320370767017758930737475134873102444517326782855384592286)
I really know the value of d! (≥∇≤)
Do you trust me? 0v0
>
```

• You would start from point Q like this; Q is random every time

The challenge is split into two phases:

1. The Flawed Proof: The server offers to "prove" it knows d through an interactive protocol. This protocol contains a critical vulnerability related to improper input validation, exposing a powerful calculation oracle.
2. The Real Proof: After the player is convinced (or has exploited the flaw), the roles are reversed. The player must prove they know d by completing 10 rounds of a Schnorr-like identification protocol.

The goal is to exploit the vulnerability in Phase 1 to recover the secret key d, then use d to pass the proofs in Phase 2 within a 30-second time limit to receive the flag.

Detailed Walkthrough

Step 1: Initial Analysis and Reconnaissance

The provided source code consists of two files: SquigglyStuff.py and chal.py.

- **SquigglyStuff.py**: This is a standard implementation of ECC point operations. It defines a Point class and a Squiggle class which handles point addition, negation, and scalar multiplication (mul). The key parameters are initialized in chal.py to match secp256k1:
  - a = 0, b = 7
  - p = 2**256 - 2**32 - 2**9 - 2**8 - 2**7 - 2**6 - 2**4 - 1
- **chal.py**: This file contains the main challenge logic.
  - A secret d is randomly generated.
  - The public key Q = d * G is calculated and sent to the player.
  - The first loop begins if the player answers anything other than 'y' to the trust prompt. Inside this loop, the server accepts two points, C1 and C2, and returns the result of squiggle.add(C2, squiggle.mul(-d, C1)), which is mathematically C2 - d*C1.
  - The crucial flaw is that the deserialize_point function and the Squiggle methods **do not check if the provided points are on the curve**. This is the entry point for our attack.

Critical Flaw: Lack of Curve Membership Verification (□□□)

The vulnerability is a classic input validation bypass. The server's deserialize_point function and the Squiggle methods blindly accept any provided (x, y) coordinates. There is no check to confirm if these coordinates satisfy the secp256k1 curve equation $y^2 \equiv x^3 + 7 \pmod p$. This allows an attacker to submit points that do not lie on the specified curve but are valid on a different, cryptographically weak curve.

This flaw effectively exposes a powerful computational oracle. The server provides a function f(C1, C2) = C2 - d*C1. By strategically choosing our

inputs, we can manipulate this oracle. Specifically, by setting C2 to the point at infinity (the identity element), the oracle simplifies to f(C1, infinity) = -d*C1. This provides us with a "black box" that performs scalar multiplication of the secret key d with any point C1 of our choosing.

Step 2: The Flawed Path - Invalid Curve Attack (CRT) /(ㅜ o ㅜ)/~~

My first instinct was to use this oracle to perform a standard Invalid Curve Attack with the goal of using the Chinese Remainder Theorem (CRT). This approach, while theoretically sound, failed in this specific challenge due to two critical limitations:

- **Curve Parameter Mismatch:** The server's computation engine uses a=0 in its multiplication formula. For the required property d*P = (d mod n)*P to hold, the custom curve on which P has a small order n must *also* have a=0. This significantly restricts the pool of usable points for the attack.
- **Round Limitation:** The secret key d is a 256-bit integer. Recovering it via the Chinese Remainder Theorem (CRT) would require collecting congruences for numerous small primes whose product exceeds 2^256. The server's hard limit of 10 interaction rounds makes it mathematically impossible to gather sufficient information.

The failure of this method indicated that a more efficient, single-shot attack was necessary.

Step 3: The "Aha!" Moment - The Singular Curve Attack (o゜v゜)ノ

If a CRT-based attack is too slow, the vulnerability must allow for a more direct extraction of d. This leads to the powerful **Singular Curve Attack**.

A singular elliptic curve is a "degenerate" form where the Elliptic Curve Discrete Logarithm Problem (ECDLP) is easy to solve. A curve $y^2 = x^3 + ax + b$ is singular if its discriminant $4a^3 + 27b^2$ is zero.

- The server's computation engine uses a = 0.
- To make the discriminant zero, we just need $27b^2 = 0$, which means b must be **0**.

This gives us our target: the singular curve $\mathbf{y^2 = x^3}$.

The magic of this curve is that its group of points is isomorphic (mathematically equivalent) to a simple additive group over the field F_p. The mapping phi that transforms the problems is:

phi(Point(x, y)) = x / y (mod p)

This mapping has a wonderful property: phi(k * P) = k * phi(P) (mod p). This means the hard ECDLP (R = k*P) becomes a simple linear equation (phi(R) = k * phi(P)), which can be solved for k with a single division! W(゜Д゜)w

Step 4: The Final Exploit

The plan was now clear and could be executed in a single shot:

1. **Choose a Point:** Select a simple point on our singular curve $y^2 = x^3$. I chose P = Point(4, 8) since $8^2 = 64$ and $4^3 = 64$.
2. **Craft the Input:** Send C1 = Point(4, 8) and C2 = Point(infinity) to the server.
3. **Trigger the Oracle:** The server, without validation, takes our point and calculates R = -d * P using its internal logic. Since its logic uses a=0 and our point's "native" curve also has a=0, the calculation is performed perfectly on our singular curve.
4. **Receive the Result:** The server returns the point R.
5. **Solve for d:**
    - We now have the equation R = -d * P on the singular curve.
    - Apply the mapping: phi(R) = phi(-d * P).
    - Using the isomorphism property: phi(R) = -d * phi(P) (mod p).
    - Let t_r = R.x / R.y and t_p = P.x / P.y.
    - The equation becomes t_r ≡ -d * t_p (mod p).
    - Solving for d: d ≡ -t_r * $(t_p)^{-1}$ (mod p).
    - This one calculation reveals the exact value of d.
6. **Pass Phase 2:** With d in hand, Phase 2 is trivial. For each of the 10 rounds, we receive C1 and C2. The required "Secret" point is simply P = C2 - d*C1, which we can now easily calculate.

The whole process is automated in a Python script since I was too lazy to test it and I wanted to get the answer automatically. (•̀ω•́)

Final Exploit Script

```
from pwn import *
import re

#
==========================================================
====================
# 1. Point Class: Defines an Elliptic Curve Point
#
==========================================================
====================

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def is_infinity(self): return self.x is None and self.y is None
    def __eq__(self, other):
        if self.is_infinity() and other.is_infinity(): return True
        if self.is_infinity() or other.is_infinity(): return False
        return self.x == other.x and self.y == other.y
    def __repr__(self):
        if self.is_infinity(): return "Point(infinity)"
        return f"Point({self.x}, {self.y})"
    def __hash__(self):
        if self.is_infinity(): return hash((None, None))
        return hash((self.x, self.y))

#
==========================================================
====================
# 2. Squiggle Class: Implements ECC Operations (Point Addition & Scalar
Multiplication)
#
==========================================================
====================
```

```
class Squiggle:       # Too lazy to write annotations (。・∀・)ノ゛
    def __init__(self, a, b, p): self.a, self.b, self.p = a, b, p
    def normalize(self, P):
        if P.is_infinity(): return P
        return Point(P.x % self.p, P.y % self.p)
    def add(self, P, Q):
        P, Q = self.normalize(P), self.normalize(Q)
        if P.is_infinity(): return Q
        if Q.is_infinity(): return P
        if P == self.negate(Q): return Point(None, None)
        if P == Q:
            if P.y == 0: return Point(None, None)
            t = (3 * P.x * P.x + self.a) * pow(P.y << 1, -1, self.p) % self.p
        else:
            if P.x == Q.x: return Point(None, None)
            t = (Q.y - P.y) * pow(Q.x - P.x, -1, self.p) % self.p
        x = (t * t - P.x - Q.x) % self.p
        y = ((P.x - x) * t - P.y) % self.p
        return Point(x, y)
    def negate(self, P):
        P = self.normalize(P)
        if P.is_infinity(): return P
        return Point(P.x, -P.y % self.p)
    def mul(self, k, P):
        P = self.normalize(P)
        result = Point(None, None)
        current = P
        if k < 0: return self.mul(-k, self.negate(P))
        while k > 0:
            if k & 1: result = self.add(result, current)
            current = self.add(current, current)
            k >>= 1
        return result

def deserialize_point(s):
    s = s.strip()
    if s == "Point(infinity)": return Point(None, None)
    try:
```

```
        coords = s[6:-1].split(", ")
        return Point(int(coords[0]), int(coords[1]))
    except:
        raise ValueError(f"Invalid point format: {s}")

def modinv(a, m):
    return pow(a, -1, m)

# ============================================================================
# 3. Standard secp256k1 Curve Parameters
# ============================================================================
p = 2**256 - 2**32 - 2**9 - 2**8 - 2**7 - 2**6 - 2**4 - 1
order = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
G = Point(0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798, 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8)
server_squiggle = Squiggle(0, 7, p) # The curve equation is y^2 = x^3 + 7 (mod p). a=0, b=7.

# ============================================================================
# 4. Phase 1: Key Recovery via Singular Curve Attack (The Exploit)
# ============================================================================
conn = remote('chall.25.cuhkctf.org', 25059)    #It's a remote connection to the challenge server. Auto [HACKING] LOL ^_____^
conn.recvuntil(b'Q: ')
q_str = conn.recvline().strip().decode()
```

```
Q = deserialize_point(q_str)
log.info(f"Received Public Key Q: {Q}")

log.info("Starting Phase 1: Singular Curve Attack")
P_singular = Point(4, 8)
log.info(f"Using singular point P = {P_singular}")
conn.sendlineafter(b'> ', b'n')
conn.sendlineafter(b'C1: ', str(P_singular).encode())
conn.sendlineafter(b'C2: ', b'Point(infinity)')
conn.recvuntil(b'The thing you need is ')
result_str = conn.recvline().strip().decode()
R = deserialize_point(result_str)
log.info(f"Received result R = {R}")
t_p = (P_singular.x * modinv(P_singular.y, p)) % p
t_r = (R.x * modinv(R.y, p)) % p
d = (-t_r * modinv(t_p, p)) % p
log.success(f"Recovered secret key d = {d}")

# Verification: Ensures the recovered key is correct
if server_squiggle.mul(d, G) == Q:
    log.success("Verification successful: d * G == Q")
else:
    log.error("Verification failed: d * G != Q. Something went wrong.")
    exit()

#
============================================================
====================
# 5. Phase 2: Proving Knowledge of 'd' (The Final Task)
#
============================================================
====================
log.info("Starting Phase 2: Proving knowledge of 'd'")
conn.sendlineafter(b'> ', b'y') # Denotes the second part of the challenge

for i in range(10): # The server requires 10 rounds of proof
    log.info(f"Starting proof round {i + 1}")
    conn.recvuntil(b'C1: ')
```

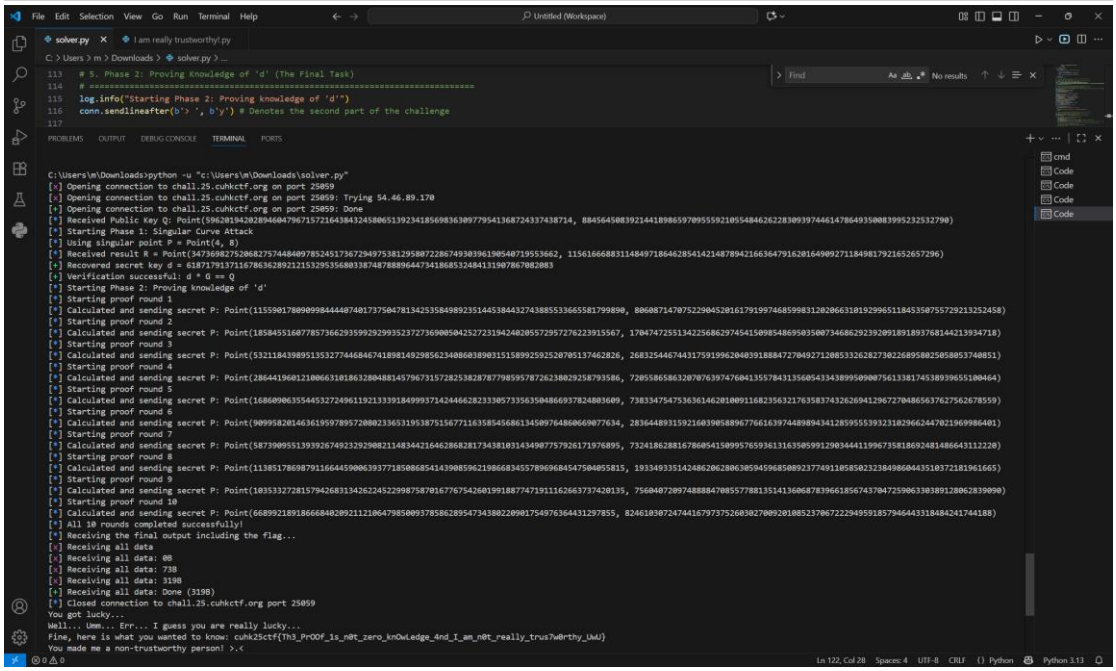
```
    C1 = deserialize_point(conn.recvline().strip().decode()) # Receive a
random point C1
    conn.recvuntil(b'C2: ')
    C2 = deserialize_point(conn.recvline().strip().decode()) # Receive
another random point C2 (which is C1 + d*P for some unknown P)

    d_C1 = server_squiggle.mul(d, C1)
    neg_d_C1 = server_squiggle.negate(d_C1)
    secret_P = server_squiggle.add(C2, neg_d_C1)

    log.info(f"Calculated and sending secret P: {secret_P}")
    conn.sendlineafter(b'Secret: ', str(secret_P).encode()) # Send the
recovered P_secret

log.info("All 10 rounds completed successfully!")

log.info("Receiving the final output including the flag...")
final_output = conn.recvall(timeout=2) # Read the remaining output (which
contains the flag)
print(final_output.decode(errors='ignore'))
conn.close()
```



• Effects in Visual Studio Code

Conclusion

This was a fantastic challenge that perfectly illustrates a common but critical implementation flaw in cryptographic systems: **input validation is paramount**. By failing to verify that the user-supplied points were on its own curve, the server allowed us to completely change the underlying mathematical problem from a secure one (ECDLP on secp256k1) to a broken one (ECDLP on a singular curve), leading to a full recovery of the private key.

Hints : You can use AI to easily get ideas about SquigglyStuff.py and chal.py and understand what you need to do ○(＜皿＜)っ Hehehe…

Flag:

cuhk25ctf{Th3_PrOOf_1s_n0t_zero_knOwLedge_4nd_I_am_n0t_really_trus7w0rthy_UwU}

Reference

Elliptic Curve Cryptography (ECC) : *https://cryptobook.nakov.com/asymmetric-key-ciphers/elliptic-curve-cryptography-ecc*

ECC Encryption / Decryption : *https://cryptobook.nakov.com/asymmetric-key-ciphers/ecc-encryption-decryption*

Download

https://drive.google.com/drive/folders/1yx12g5_RjEAkkYkMFYUpyclAEcmHVTP5?usp=drive_link